

## Laborator 10

# Probleme de sincronizare

### 10.1 Scopul lucrării

Laboratorul curent este unul recapitulativ, în care se folosesc conceptele acumulate în laboratoarele anterioare pentru a rezolva unele probleme de sincronizare mai complexe.

### 10.2 Mersul lucrării

#### 10.2.1 Problema bărbierului

Se va simula salonul unui bărbier, în care mai mulți clienți sosesc pe rând și sunt serviți, respectându-se următoarele cerințe:

- salonul are `NR_CHAIRS` scaune în sala de așteptare, plus scaunul bărbierului, pe care se așază clientul care e servit;
- clienții sosesc în salon la momente de timp aleatorii, iar dacă
  - sunt scaune disponibile în sala de așteptare, clientul așteaptă să îi vină rândul;
  - toate scaunele din sala de așteptare sunt ocupate, clientul decide că e prea aglomerat și pleacă;
- atunci când scaunul bărbierului e liber, clientul care așteaptă de cel mai mult timp se așază pe el și e servit de către bărbier;
- atunci când nu are clienți, bărbierul doarme și este trezit de clienți atunci când aceștia trebuie serviți;
- atât bărbierul cât și cei `NR_CUSTOMERS` clienți vor fi simulați printr-un thread separat.

Se vor implementa două soluții: prima dintre ele, în fișierul `l10p1_barber_sem.c` se bazează pe semafoare, în timp ce a doua soluție, din fișierul `l10p2_barber_mutex_cond.c` se bazează pe lacăte și variabile condiționale.

##### 10.2.1.1 Soluția 1: semafoare

```
1 | #define NR_CUSTOMERS 20
2 | #define NR_CHAIRS 3
3 |
4 | sem_t lock;
5 | sem_t customer;
6 | sem_t barber;
7 | int waitingCustomers = 0;
8 | int barberChair = 0;
```

Vom începe prin a defini constantele problemei, respectiv numărul de clienți care vin la salon de-a lungul zilei `NR_CUSTOMERS`, respectiv numărul de scaune disponibile în sala de așteptare `NR_CHAIRS`.

Numărul de clienți care se află la un moment dat în sala de așteptare se va reprezenta prin variabila globală `waitingCustomers`. Accesul la această variabilă va fi sincronizat prin semaforul `lock`.

Clienții așteaptă să fie serviți prin semaforul `customer`, iar bărbierul așteaptă clienții prin semaforul `barber`. Variabila `barberChair` indică id-ul clientului care se află la un moment dat pe scaunul bărbierului.

```

1  int main()
2  {
3      int i;
4      pthread_t tidC[NR_CUSTOMERS], tidB;
5
6      srand(time(NULL));
7
8      sem_init(&lock, 0, 1);
9      sem_init(&customer, 0, 0);
10     sem_init(&barber, 0, 0);
11
12     pthread_create(&tidB, NULL, th_barber, NULL);
13     for(i=0; i< NR_CUSTOMERS; i++) {
14         pthread_create(&tidC[i], NULL, th_customer, (void*)(ssize_t)(i+1));
15     }
16     for(i=0; i< NR_CUSTOMERS; i++) {
17         pthread_join(tidC[i], NULL);
18     }
19     pthread_cancel(tidB);
20     pthread_join(tidB, NULL);
21
22     sem_destroy(&lock);
23     sem_destroy(&customer);
24     sem_destroy(&barber);
25
26     return 0;
27 }
```

În funcția `main()` vom defini variabile pentru thread id-urile clienților și pentru thread id-ul bărbierului (linia 4). La linia 6 se inițializează generatorul de numere aleatoare, pe baza timpului curent. Cele trei semafoare utilizate se vor inițializa la începutul programului (liniile 8–10) și se vor distruge la final (liniile 22–24).

La linia 12 se creează thread-ul bărbierului iar la liniile 13–15 se creează thread-urile clienților.

La liniile 16–18 se așteaptă terminarea thread-urilor clienților. Thread-ul bărbierului va executa o buclă infinită (codul lui va fi detaliat mai jos), așa că trebuie oprit explicit după terminarea thread-urilor clienților, apelând `pthread_cancel()` (linia 19). Funcția `pthread_cancel()` nu oprește thread-ul imediat, doar îi transmite semnalul de oprire. La linia 20 se așteaptă oprirea acestuia.

Fiecare thread client trebuie să primească ca argument propriul id (un număr cuprins între 1 și `NR_CUSTOMERS`). Cum variabila `i` iterează între 0 și `NR_CUSTOMERS - 1`, vom transmite valoarea `i + 1`. Funcția `pthread_create()` se așteaptă ca argumentul transmis să fie de tipul `void*`, așa că trebuie să facem cast la acel tip. Dacă am fi făcut cast-ul direct, de la `int` la `void*` am fi primit următorul warning, pe un sistem pe 64 bit: **cast to pointer from integer of different size**. Acest warning ne spune că tipul `int` este pe 32 bit, în timp ce un pointer e pe 64 bit. Din acest motiv, se vor face două cast-uri: întâi de la `int` la `ssize_t` (care este un tip de date numeric, de aceeași dimensiune cu cea a unui pointer), apoi de la `ssize_t` la `void*`.

În continuare, vom folosi un mecanism similar cu cel din problema *rendezvous*.

```

1 void *th_barber(void *arg)
2 {
3     for(;;) {
4         sem_post(&customer);
5         sem_wait(&barber);
6         printf("[B ] Serving customer %d\n", barberChair);
7         usleep(1000);
8     }
9     return NULL;
10 }

```

Thread-ul bărbierului va executa o buclă infinită, în care va aștepta venirea clienților și îi va servi. Acest thread nu se va termina niciodată singur, dar va fi oprit de thread-ul principal, prin apelul funcției `pthread_cancel()`.

La linia 4, se dă o permisiune semaforului `customer`, indicând faptul că scaunul bărbierului e liber și se așteaptă clienți. La linia 5, se așteaptă după semaforul `barber`. Acest semafor va primi permisiuni de la clienți, atunci când aceștia se așază pe scaunul bărbierului. Atunci când ajungem la liniile 6–7 înseamnă că semaforul `barber` a primit o permisiune, deci avem un client de servit.

```

1 void *th_customer(void *arg)
2 {
3     int myId = (int)(ssize_t)arg;
4     int tooBusy = 0;
5
6     usleep(1000 * (rand() % 20));
7     printf("[C%02d] Entering the barber shop\n", myId);
8
9     sem_wait(&lock);
10    if(waitingCustomers < NR_CHAIRS) {
11        ++waitingCustomers;
12        printf("[C%02d] %d customer(s) waiting\n", myId, waitingCustomers);
13    } else {
14        tooBusy = 1;
15        printf("[C%02d] Too busy, will come back another day.\n", myId);
16    }
17    sem_post(&lock);
18
19    if(!tooBusy) {
20        sem_wait(&customer);
21
22        sem_wait(&lock);
23        --waitingCustomers;
24        sem_post(&lock);
25
26        barberChair = myId;
27        sem_post(&barber);
28        printf("[C%02d] being served\n", myId);
29        usleep(1000);
30    }
31
32    return NULL;
33 }

```

Thread-urile clienților vor începe prin obținerea id-ului propriu, făcând cast argumentului `arg` (linia 3).

Fiecare client va aștepta un timp aleatoriu (linia 6), apoi își va anunța sosirea în salon (linia 7).

Mai departe, trebuie verificată variabila `waitingCustomers`, pentru a decide dacă mai este loc în sala de așteptare. Accesul la această variabilă va fi protejat de semaforul `lock` (liniile 9–17).

Dacă numărul de clienți din sala de așteptare este mai mic decât `NR_CHAIRS`, se ia loc în sala de așteptare, incrementând variabila `waitingCustomers` (linia 11) și se afișează câți clienți sunt în așteptare (linia 12). Altfel, se setează variabila locală `tooBusy` și se afișează că salonul e prea aglomerat (linia 15). Observați că variabila `tooBusy` este o variabilă locală, fiecare thread având propria “versiune” a acelei variabile. Prin urmare, nu există acces concurent la acea variabilă între thread-uri, motiv pentru care valoarea ei poate fi testată și după eliberarea semaforului `lock`.

După ce am ieșit din regiunea critică, se verifică variabila `tooBusy`. Dacă acesta nu a fost setată (a rămas 0) așteptăm să se elibereze scaunul bărbierului, așteptând după semaforul `customer` (linia 20). După ce trecem de acest semafor decrementăm variabila `waitingCustomers`, protejând accesul la variabilă cu semaforul `lock` (liniile 22–24). Decrementarea acestei variabile indică faptul că s-a părăsit sala de așteptare și urmează să fim serviți de bărbier. În acest punct avem acces exclusiv la variabila `barberChair`, pe care o setăm la id-ul thread-ului curent, apoi dăm o permisiune semaforului `barber` (linia 27), care va trezi bărbierul, iar clientul curent va fi servit.

### 10.2.1.2 Soluția 2: lacăte și variabile condiționale

Vom prezenta o soluție similară cu cea precedentă, dar vom utiliza lacăte și variabile condiționale în loc de semafoare.

```

1  #define NR_CUSTOMERS 20
2  #define NR_CHAIRS 3
3
4  pthread_mutex_t lock;
5  pthread_cond_t barber;
6  pthread_cond_t customer;
7  int waitingCustomers = 0;
8  int barberChair = 0;
```

Lacătul `lock` va proteja regiunile critice, în timp ce variabilele condiționale `barber` și `customer` vor fi folosite pentru a pune thread-urile în așteptare. Variabila `barberChair` indică id-ul clientului care se află la un moment dat în scaunul bărbierului, sau 0 dacă scaunul e liber.

```

1  int main()
2  {
3      int i;
4      pthread_t tidC[NR_CUSTOMERS], tidB;
5
6      srand(time(NULL));
7
8      pthread_mutex_init(&lock, NULL);
9      pthread_cond_init(&customer, NULL);
10     pthread_cond_init(&barber, NULL);
11
12     pthread_create(&tidB, NULL, th_barber, NULL);
13     for(i=0; i< NR_CUSTOMERS; i++) {
14         pthread_create(&tidC[i], NULL, th_customer, (void*)(size_t)(i+1));
15     }
16     for(i=0; i< NR_CUSTOMERS; i++) {
17         pthread_join(tidC[i], NULL);
18     }
19     pthread_cancel(tidB);
20     pthread_join(tidB, NULL);
21
22     pthread_mutex_destroy(&lock);
23     pthread_cond_destroy(&customer);
24     pthread_cond_destroy(&barber);
25
26     return 0;
```

```
27 | }
```

Funcția `main()` este similară cu cea din soluția anterioară, dar în loc de semafoare inițializăm (liniile 8–10) și distrugem (liniile 22–24) lacătul și variabilele condiționale declarate anterior.

```
1 void *th_barber(void *arg)
2 {
3     int crtCustomer = 0;
4     for(;;) {
5         pthread_mutex_lock(&lock);
6         while(barberChair == 0){
7             pthread_cond_wait(&barber, &lock);
8         }
9         crtCustomer = barberChair;
10        pthread_mutex_unlock(&lock);
11
12        printf("[B ] Serving customer %d\n", crtCustomer);
13        usleep(1000);
14
15        pthread_mutex_lock(&lock);
16        barberChair = 0;
17        pthread_cond_signal(&customer);
18        pthread_mutex_unlock(&lock);
19    }
20    return NULL;
21 }
```

Bărbierul va executa o buclă infinită, la fel ca la soluția anterioară.

Vom proteja variabila `barberChair` prin lacătul `lock`. Cât timp aceasta are valoarea 0 (nu avem clienți pe scaun), bărbierul trebuie să doarmă (liniile 6–8).

După ce avem un client care s-a așezat pe scaun, îi reținem id-ul (linia 9), apoi ieșim din regiunea critică (linia 10) și îl servim (liniile 12–13).

După servirea clientului, setăm variabila `barberChair` pe 0 (s-a eliberat scaunul), apoi semnalizăm eventualului client în așteptare.

```
1 void *th_customer(void *arg)
2 {
3     int myId = (int)(size_t)arg;
4     int tooBusy = 0;
5
6     usleep(1000 * (rand() % 20));
7     printf("[C%02d] Entering the barber shop\n", myId);
8
9     pthread_mutex_lock(&lock);
10    if(waitingCustomers < NR_CHAIRS) {
11        ++waitingCustomers;
12        printf("[C%02d] %d customer(s) waiting\n", myId, waitingCustomers);
13        while(barberChair != 0){
14            pthread_cond_wait(&customer, &lock);
15        }
16        --waitingCustomers;
17        barberChair = myId;
18        pthread_cond_signal(&barber);
19    } else {
20        tooBusy = 1;
21        printf("[C%02d] Too busy, will come back another day.\n", myId);
22    }
23    pthread_mutex_unlock(&lock);
```

```

24 |
25 |     if(!tooBusy) {
26 |         printf("[C%02d] being served\n", myId);
27 |         usleep(1000);
28 |     }
29 |
30 |     return NULL;
31 | }

```

La fel ca la soluția anterioară, clienții își obțin propriul id (linia 3), așteaptă un timp random (linia 6), apoi verifică numărul de locuri ocupate din sala de așteptare (linia 10).

Dacă au fost suficiente locuri în sala de așteptare, incrementăm variabila `waitingCustomers` și afișăm valoarea acesteia. În acest punct clientul a ajuns în sala de așteptare și trebuie să aștepte eliberarea scaunului bărbierului (valoarea variabilei `barberChair` trebuie să devină 0). După ce a trecut de buclă de la liniile 13–15, clientul se poate așeza pe scaunul bărbierului. Se va elibera scaunul din sala de așteptare (linia 16), se va ocupa scaunul bărbierului (linia 17), apoi acesta va fi trezit (linia 18).

### 10.2.2 Probleme propuse

- (25p) În soluțiile prezentate în secțiunile 10.2.1.1 și 10.2.1.2 bărbierul execută o buclă infinită și este oprit de threadul principal prin funcția `pthread_cancel()`. Modificați exemplele date, astfel încât thread-ul bărbierului să se închidă singur, după ce toți cei `NR_CUSTOMER` clienți și-au executat thread-ul.
- (25p) Servirea propriu-zisă a clienților începe sincronizat (thread-ul bărbierului și thread-ul clientului se sincronizează la intrarea în această etapă), dar clientul ar putea de exemplu să “părăsească scaunul” înainte ca bărbierul să termine. Modificați unul din cele două exemple date, astfel încât clientul servit să nu își încheie thread-ul până nu este anunțat de bărbier că a terminat.
- (50p) **Problema mesei de restaurant:** La o masă de restaurant au loc 5 persoane. Dacă o persoană ajunge la restaurant și sunt locuri libere la masă, se poate așeza imediat. Dacă, în schimb, întreaga masă e ocupată, înseamnă că persoanele de la masă formează un grup și trebuie să aștepte ca toate persoanele să plece de la masă, ca să se poată așeza. Fiecare persoană stă la masă un timp random.
- (50p) **Traversarea râului:** Pe marginea Someșului se află bărci care au o capacitate de 4 persoane, și care pot fi folosite doar atunci când sunt pline. Avem două tipuri de persoane, care se vor simula prin thread-uri: suporter “U” și suporter “CFR”. Din motive de siguranță, nu este permis ca într-o barcă să se găsească un suporter al unei echipe împreună cu 3 suporter ai celeilalte echipe (deci fie avem 2-2, fie 4-0). Nu se va începe îmbarcarea într-o barcă, până ce barca anterioară nu s-a umplut.
- (60p) **Problema fumătorilor:** Pentru ca o persoană să fumeze are nevoie de 3 ingrediente: tutun, hârtie și brichetă. Există 3 fumători, fiecare dintre ei având o cantitate nelimitată din unul din ingrediente, și un “dealer” (furnizor), care la momente random de timp oferă câte două ingrediente random. De fiecare dată când dealer-ul oferă cele două ingrediente, fumătorul care are ingredientul complementar trebuie să le ia. Cele 3 ingrediente vor fi simulate prin 3 semafoare, iar dealer-ul și cei 3 fumători prin câte un thread. Dealer-ul trebuie doar să acorde permisiuni la cele două semafoare alese random, nu are voie să execute alte acțiuni.